

TÓM TẮT PYTORCH + NLP CƠ BẢN

Từ Tensor, Autograd, Linear Layer đến Embedding, Batching và Word Window Classifier

1. Tensor và indexing

Tensor là cấu trúc dữ liệu số của PyTorch. Với tensor 2 chiều, có thể nhớ cú pháp: data[hàng, cột].

```
data[:, 0] # tất cả hàng, cột đầu tiên
data[0, :] # hàng đầu tiên, tất cả cột
data[:, 1] # tất cả hàng, cột thứ 2
```

Python/PyTorch đánh index từ 0.

2. Autograd, backward và gradient

```
x = torch.tensor(2.0, requires_grad=True)
z = x * x * 3 # z = 3x^2
z.backward()
print(x.grad) # dz/dx = 6x = 12
```

backward() tính gradient. Gradient được lưu trong .grad. PyTorch mặc định cộng dồn gradient, vì vậy trước vòng tính mới thường cần xóa gradient cũ.

```
optimizer.zero_grad() # xóa gradient cũ
loss.backward() # tính gradient
optimizer.step() # cập nhật parameter
```

3. Linear Layer

nn.Linear(in_features, out_features) thực hiện phép biến đổi tuyến tính $y = Wx + b$.

```
layer = nn.Linear(4, 2)
output = layer(x)
```

Linear(4, 2) nhận 4 features và trả 2 features. Weight có shape (2,4), bias có shape (2,). requires_grad=True nghĩa là các parameter này được tính gradient khi training.

list(layer.parameters()) chỉ hiển thị weight và bias hiện tại; còn layer(x) mới thực hiện tính toán.

4. Activation và nn.Sequential

Sigmoid biến từng giá trị về khoảng 0 đến 1 và tạo tính phi tuyến.

```
sigmoid = nn.Sigmoid()
output = sigmoid(linear_output)
```

nn.Sequential gom nhiều layer thành một chuỗi xử lý.

```
block = nn.Sequential(
    nn.Linear(4, 2),
    nn.Sigmoid()
)
output = block(input)
```

Luồng: Input -> Linear -> Sigmoid -> Output.

5. Tự tạo model bằng nn.Module

Muốn tự xây layer/model, tạo class kế thừa nn.Module. Có hai phần quan trọng:

__init__(): model có những layer/parameter gì.

forward(x): dữ liệu x đi qua các layer theo đường nào.

```
class Model(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(4, 2)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.linear(x)
        x = self.sigmoid(x)
        return x
```

Có thể dùng Sequential để viết ngắn khi các layer chạy tuần tự. Khai báo từng layer riêng linh hoạt hơn cho model có nhiều nhánh hoặc đường đi phức tạp.

6. MLP ví dụ

Một Multilayer Perceptron đơn giản:

```
Input
-> Linear(input_size, hidden_size)
-> ReLU
-> Linear(hidden_size, input_size)
-> Sigmoid
-> Output
```

model(input) gọi forward và cho input chạy qua model. list(model.named_parameters()) xem tên cùng weight/bias mà model sẽ học. ReLU và Sigmoid không có weight/bias.

7. Optimizer và Loss

Gradient chỉ cho biết parameter nên thay đổi thế nào; optimizer mới thực sự cập nhật parameter.

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.1)
```

model.parameters(): các weight/bias cần tối ưu. lr: learning rate, quyết định kích thước bước cập nhật. SGD và Adam là hai optimizer phổ biến.

Loss đo mức sai khác giữa dự đoán và target. Ví dụ MSE:

```
y_pred = model(x)
loss = nn.MSELoss()(y_pred, y)
```

Loss càng giảm thì dự đoán thường càng gần target. MSE không phải phần trăm sai.

8. NLP preprocessing và vocabulary

```
def preprocess_sentence(sentence):
    return sentence.lower().split()
```

Preprocessing ở đây: chuyển chữ thường rồi tokenize thành danh sách từ.

```
vocabulary = set(w for s in train_sentences for w in s)
```

vocabulary chứa tất cả từ duy nhất trong corpus.

<unk>: đại diện từ không có trong vocabulary. <pad>: vị trí trống được thêm vào để padding/context window.

9. Word -> Index

```
ix_to_word = sorted(list(vocabulary))
word_to_ix = {word: ind for ind, word in enumerate(ix_to_word)}
```

word_to_ix gán mỗi từ một ID/index duy nhất; không phải đếm tần suất.

```
"paris" -> 10
"love" -> 5
```

Nếu token không tồn tại, dùng index của <unk>. ix_to_word làm chiều ngược lại: index -> word.

10. Embedding

Index chỉ là ID, chưa biểu diễn ý nghĩa của từ. nn.Embedding ánh xạ mỗi index thành một vector có thể học.

```
embedding_dim = 5
embeds = nn.Embedding(len(vocabulary), embedding_dim)
paris_vector = embeds(torch.tensor(word_to_ix['paris']))
```

Ví dụ: paris -> index 10 -> vector 5 chiều. embedding_dim là hyperparameter do người xây model chọn; 5 chỉ là ví dụ đơn giản.

11. Batching và DataLoader

Không cập nhật sau toàn bộ corpus và cũng không nhất thiết cập nhật sau từng mẫu. Ta gom nhiều mẫu thành batch để training ổn định và hiệu quả hơn.

```
loader = DataLoader(
    data,
    batch_size=2,
    shuffle=True,
    collate_fn=collate_fn
)
```

batch_size: số mẫu mỗi batch. shuffle=True: xáo trộn dữ liệu. collate_fn: xử lý thêm từng batch trước khi đưa vào model.

Trong bài này collate_fn: window padding -> token thành index -> pad các câu/label về cùng chiều dài -> lưu độ dài thật của từng câu.

12. Context Window và unfold

Với window_size = N, mỗi window có $2N + 1$ token: N từ bên trái + từ trung tâm + N từ bên phải.

```
window_size = 2
# mỗi window có 5 token
windows = inputs.unfold(1, 5, 1)
```

Ví dụ input đã pad: [0,0,9,7,8,18,0,0] sẽ được tách thành các cửa sổ 5 token trượt từng bước 1. Mỗi window dùng để dự đoán nhãn của từ ở giữa.

13. WordWindowClassifier

Mục tiêu: dự đoán từng từ có phải LOCATION hay không.

```
Token indices
-> unfold tạo windows
-> Embedding
-> ghép embeddings của window
-> Linear + Tanh
-> Linear
-> Sigmoid
-> P(LOCATION)
```

Nếu window_size=2 và embed_dim=25: mỗi window có 5 từ, nên sau embedding có $5 \times 25 = 125$ giá trị. Hidden layer biến 125 -> hidden_dim. Output layer biến hidden_dim -> 1. Sigmoid đưa kết quả về 0-1.

14. Training loop - phần quan trọng nhất

```
for inputs, labels, lengths in loader:
    optimizer.zero_grad()    # 1. xóa gradient cũ
```

```
outputs = model(inputs)    # 2. forward
loss = loss_function(outputs, labels, lengths) # 3. loss
loss.backward()            # 4. tính gradient
optimizer.step()           # 5. cập nhật parameter
```

Một epoch = model đi qua toàn bộ training dataset một lần. Training nhiều epoch lặp lại chu trình trên.

15. Test

```
test = "She comes from Paris"
labels = [0, 0, 0, 1]
# 0 = không phải LOCATION
# 1 = LOCATION
```

Sau preprocessing và padding, câu test được đưa qua cùng model. Output có thể dạng [0.02, 0.04, 0.08, 0.95], nghĩa là model cho Paris xác suất LOCATION cao.

16. Khung tổng thể cần nhớ

Giai đoạn	Luồng
Chuẩn bị NLP	Sentence -> lowercase/tokenize -> vocabulary -> index -> padding
Biểu diễn từ	Word -> index -> embedding vector
Model	Window -> Embedding -> Linear/Tanh -> Linear/Sigmoid
Training	zero_grad -> forward -> loss -> backward -> optimizer.step
Test	Input mới -> preprocessing -> model -> prediction

Câu chốt: Neural network nhận tensor số, thực hiện forward để dự đoán, loss đo sai số, backward tính gradient và optimizer dùng gradient để cập nhật các parameter. Trong NLP, trước đó ta cần biến từ thành index rồi embedding vector để model có thể xử lý.